

编译原理实验报告：语义分析

一、实验目的

- 理解编译器前端中语义分析阶段的作用与地位。
- 掌握符号表的构建与管理方法，包括变量、函数、结构体类型的存储与查询。
- 实现类型检查与类型等价性判定规则（结构等价、名字等价）。
- 能够检测并报告常见的语义错误，如未定义标识符、类型不匹配、重复定义等。
- 通过实现一个完整的语义分析器，加深对属性文法、语法制导翻译和符号表设计的理解。

二、实验内容

本次实验基于一个类 C 语言的简单文法，在语法分析生成的抽象语法树（AST）之上进行语义分析。主要任务包括：

- 设计并实现符号表数据结构，支持多级作用域（全局与函数局部作用域）。
- 实现类型系统，支持基本类型（`int`、`float`）、数组类型、结构体类型、函数类型。
- 遍历 AST，对每个语法结构进行语义检查，计算表达式的类型与左值属性。
- 检测并报告 19 种预定义的语义错误（见后文错误类型表）。
- 输出错误信息，格式为：Error type <编号> at Line <行号>: <描述>。

三、系统总体设计

3.1 类型系统设计

类型用枚举 `TypeKind` 区分，类型结构 `Type` 采用带标签的联合体表示：

```
typedef enum {
    T_INT, T_FLOAT, T_ARRAY, T_STRUCT, T_FUNC, T_ERROR
} TypeKind;

typedef struct Type_ {
    TypeKind kind;
    union {
        struct { struct Type_* elem; int size; } array;
        struct { char* name; FieldList* fields; } strct;
        struct { struct Type_* ret; FieldList* params; } func;
    };
} Type;
```

- 基本类型：** `T_INT`、`T_FLOAT`
- 数组类型：** 包含元素类型 `elem` 和大小 `size`（本实现忽略大小进行类型等价比较）
- 结构体类型：** 包含名字 `name`（可为匿名）与域链表 `fields`
- 函数类型：** 包含返回类型 `ret` 和参数链表 `params`
- 错误类型 `T_ERROR`：** 用于抑制级联错误

类型等价判断函数 `typeEqual` 实现了结构等价：

- 数组忽略长度，仅比较元素类型
- 结构体按域名及顺序比较各域类型（忽略结构体名称，即结构等价）
- 函数比较返回类型及参数类型列表

3.2 符号表设计

符号表分为两个全局链表：

- `varTable`：存储变量和函数标识符，每个表项包含名字、类型、是否已定义（函数体）、首次出现行号。
- `structDefs`：存储已定义的结构体类型名（标签名），每个表项包含名字和对应的结构体类型。

符号表采用单链表实现，查找为线性搜索。作用域管理通过保存/恢复全局 `varTable` 指针实现：进入函数定义时，保存当前 `varTable`，将形参加入新作用域，分析完函数体后恢复。

3.3 主要函数模块

函数名	功能描述
<code>semanticAnalysis</code>	语义分析入口，遍历根节点 <code>ExtDefList</code> ，最后检查未定义的函数声明。
<code>analyzeExtDefList</code> / <code>ExtDef</code>	处理全局定义：变量声明、结构体定义、函数声明/定义。
<code>analyzeSpecifier</code>	将 <code>Specifier</code> AST 节点转换为 <code>Type</code> ，处理结构体定义与引用。
<code>analyzeStructSpecifier</code>	处理结构体定义（含域列表）或结构体标签引用，检查重复定义错误。
<code>analyzeVarDec</code>	分析变量声明部分（可能带数组下标），返回最终类型和变量名。
<code>analyzeCompSt</code> / <code>DefList</code> / <code>Stmt</code>	处理复合语句、局部声明、各类语句（表达式、if、while、return）。
<code>analyzeExp</code>	核心表达式类型推导函数，返回表达式类型及左值标志，进行类型检查。
<code>analyzeArgs</code>	检查函数调用实参列表与形参列表的类型及个数是否匹配。
<code>typeEqual</code> / <code>isArith</code>	类型等价判断与算术类型判断辅助函数。

四、关键算法与实现细节

4.1 类型构造与等价判断

采用结构等价策略，即两个结构体类型只要域的名字、类型、顺序相同即为同一类型，不依赖结构体标签名。对于数组，忽略数组长度，仅比较元素类型。

```
static int typeEqual(Type* a, Type* b) {
    // 错误类型匹配任何类型, 用于抑制级联错误
    if (a->kind == T_ERROR || b->kind == T_ERROR) return 1;
    if (a->kind != b->kind) return 0;
    switch (a->kind) {
        case T_ARRAY: return typeEqual(a->array.elem, b->array.elem);
        case T_STRUCT: // 遍历比较域列表
        case T_FUNC:    // 比较返回类型和参数列表
            // ...
    }
}
```

4.2 作用域管理

由于语言仅支持全局作用域和单层函数局部作用域, 作用域管理较为简单:

- 全局变量与函数名统一加入 `varTable`, 在进入语义分析前初始化。
- 遇到函数定义时:
 1. 保存当前 `varTable` 指针 (全局作用域)。
 2. 将形参列表逐个加入 `varTable`, 形成局部作用域。
 3. 分析函数体 (复合语句), 其中声明的局部变量继续加入 `varTable`。
 4. 分析完成后恢复 `varTable` 指针。

4.3 函数声明与定义的冲突处理

代码中对函数标识符采用“先声明后定义”或多次声明的灵活处理:

- 若标识符已存在且为函数类型, 检查类型是否一致 (Error 19)。
- 若一致且本次提供了函数体, 则将原符号的 `defined` 标记置为 1。
- 若已存在且已有函数体, 再次提供函数体则报告重复定义错误 (Error 4)。
- 若已存在但不是函数类型, 报告变量名冲突 (Error 3)。

最后在 `semanticAnalysis` 出口处扫描 `varTable`, 对 `defined == 0` 的函数类型符号报告 Error 18。